ILLUMINARY LABS / KNOAD

KNOAD WALLET

Permanent Ownership. Temporary Authority. Minimal Exposure.

Technical White Paper | Architecture, Authorization, Recovery, Permissions & Security Validation

Design draft 1.0 - pre-audit

13 October 2026

Research and implementation specification - NOT a security certification

Executive orientation

> **Document status.** This paper describes a proposed noncustodial EVM smart-account wallet and its required verification work. Its mechanisms are not guaranteed safe merely because they are described here. A previous research prototype exists, but the emergency recovery protocol and many integrations are not implemented or independently audited. No contract address, deployment, performance benchmark, revenue claim, or completed audit is asserted. This paper is not investment or custody advice; do not use research contracts with assets of value.

KNOAD Wallet proposes an account architecture in which the address that owns assets and identifies a user is stable, while everyday cryptographic authority is transient. The account authorizes an operation, executes the user's intended action, and—when the operation succeeds at the appropriate outer level—retire its current hot signer in favor of a freshly prepared signer. A separately isolated, user-controlled recovery authority can replace a missing or compromised hot signer without a KNOAD administrator, custodian, database or proprietary recovery service. The wallet also limits token allowances by default and makes lasting authorizations explicit rather than silently promising to revoke them all.

The motivation is not that 12- or 24-word recovery phrases are presently brute-forceable by AI. They are not, when properly generated. The immediate risks include malicious approvals, signing malware, insecure application logic, legacy contracts, and preventable wallet mistakes. Future fault-tolerant quantum computing threatens elliptic-curve signature schemes; frequently retiring ECDSA authorization can narrow one exposure window but **does not confer post-quantum security**. Ethereum's own security roadmap makes the distinction clear [R07].

Recommended architecture at the time of writing: a minimal, audited ERC-4337 account core; single live hot signer at a time; enforced signer generation and rotation; independent recovery authority derived from a user-controlled BIP-39 recovery root; **randomly generated or device-protected hot keys, with no persistent master recovery secret stored in the daily vault**; a scoped ERC-1271 policy for long-lived signatures; a bounded execution engine and policy-aware approval handling; independently recoverable transaction submission. Optional advanced BIP-39 passphrase support is offered as an explicit risk tradeoff, not a default requirement.

The development team must still resolve account factory/discovery, transaction liveness under adverse network conditions, exact marketplace signature compatibility, authentication portability, cryptographic exposure of recovery operations, module/upgrade strategy, gas budgets, and a complete audit and release program. Those are **open engineering decisions**, not claims of completed work.

Contents

PART I / THE FOUNDATION

01 Executive Summary 5

02 The Evolution of Wallet Security 7

03 Design Philosophy & Core Principles 9

PART II / THE WALLET & ITS ARCHITECTURE

04 The KNOAD Wallet Experience 10

05 Smart-Account Architecture 12

06 Cryptography & Automatic Key Rotation 14

07 Transaction Lifecycle & Reliability 16

08 Recovery & Ownership Continuity 18

09 Proactive Permission Management 20

10 Persistent Signatures & Delayed Transactions 22

PART III / USABILITY, INDEPENDENCE & ADOPTION

11 Application Design & Web3 Integration 23

12 Decentralization & Infrastructure Independence 25

PART IV / SECURITY, VERIFICATION & TRUST

13 Security Architecture & Threat Modeling 26

14 Engineering Standards, Testing & Auditing 29

PART V / SUSTAINABILITY & THE FUTURE

15 Economics & Operational Sustainability 31

16 Development Roadmap & Future Capabilities 32

17 Conclusion — The Future of Digital Ownership 33

TECHNICAL APPENDICES

A Glossary and Terminology 34

B Relevant Standards and Status 35

C Logical Contract Interfaces (Illustrative, Not Deployable) 36

D Cryptographic Derivation and Restoration Specification 37

E Canonical Flow Diagrams 38

F Security Case and Residual-Risk Register 39
G Verification and Audit Matrix 40
H Compatibility and Infrastructure Matrix 41
I Economics and Performance Measurement Plan 42
J Prototype Disposition and Open Technical Decisions 43

Reading conventions

- **Design commitment** means a product behavior chosen for the intended architecture; it does not imply a working deployment.
- **Normative requirement** uses **MUST**, **MUST NOT**, or **SHOULD** to state what a production implementation must satisfy.
- **Open design item** marks an unresolved choice that cannot safely be made by marketing or intuition.
- **Release blocker** denotes a capability or proof required before real assets may be placed in KNOAD Wallet.
- References [R01]–[R15] point to primary protocol, security, and implementation documentation at the end of the paper.

PART I — THE FOUNDATION

01. Executive Summary

1.1 Purpose

KNOAD Wallet is an effort to make blockchain ownership behave more like a durable identity and less like a permanent exposure of one long-lived signing key. Today a conventional externally owned Ethereum account (EOA) derives control from a single private key. The account address is associated with that key pair, and the owner generally protects it by safeguarding a mnemonic backup and avoiding malicious transactions. That model is simple, portable and extraordinarily resilient when correctly operated; it also offers little room to replace spending authority without moving assets to a new address.

A smart-contract account separates the **holding address** from **the secret currently allowed to spend**. Its code verifies authorization and applies policies. Under the proposed protocol, a user may own ETH, fungible tokens, NFTs, memberships and identity-linked entitlements at a stable account address while the recognized hot signing key changes after successfully processed transactions. This is not a new blockchain and not a proprietary custody system: it is a specific policy implemented on existing EVM account-abstraction infrastructure [R01].

1.2 The five product commitments

First, permanent account continuity: key changes do not migrate assets, rewrite ENS records or replace the smart account address. Second, temporary hot authority: operations are bound to an authorized signer and security generation, and routine success replaces that signer. Third, independent owner recovery: a user who still has their valid recovery material can replace an unusable hot signer through a separate onchain authorization path. Fourth, least privilege: wallet-controlled approvals prefer exact amount, short duration or atomic completion where protocols permit. Fifth, provider independence: recovery, signing and transaction submission do not depend on the continued operation of Illuminary Labs.

These commitments are bounded. The design cannot distinguish a legitimate recovery credential holder from a thief who possesses the same secrets; it cannot recover assets already transferred by an attacker; it cannot retroactively invalidate every protocol-specific offchain promise; and it cannot guarantee that a chain includes transactions during network censorship or outages.

1.3 Proposed deployment posture

KNOAD begins as a single-chain, heavily tested, open-specification smart account with a companion mobile application and desktop extension. Existing holders transfer assets into the new account themselves. The wallet does not ask for arbitrary spending approvals to perform background migrations, nor is a user required to pay KNOAD a subscription to operate or recover their account. Network fees remain a user expense unless a clearly disclosed paymaster sponsors a particular operation.

The research prototype previously discussed contains an ERC-4337 account, factory, transaction helpers, a simulated interface and state-model tests. It is not compiled, audited or verified as a production wallet. In particular it still includes a next-key proof signature that prematurely exposes that key's public-key material; it has no fully realized independent recovery; and its address prediction incorporates the initial hot signer. The production architecture specified in this document supersedes those choices. See Section 14 and Appendix J for disposition.

1.4 Success criteria

Success is measured by reproducible recovery, user-visible permission reduction, lower lifetime exposure to a previously used signer, compatibility with genuine everyday activity, transparent cost and independent audit evidence. Success is **not** demonstrated by token price, marketplace hype, a passing unit test alone, or claims that a design is “unhackable.” The engineering program must produce public verification artifacts, measurable limits and honest residual-risk disclosures.

02. The Evolution of Wallet Security

2.1 A lasting identity attached to a lasting key

EOAs excel at portability and simple cryptographic ownership. The same design constrains security upgrades: the account cannot normally revoke one private key and adopt a different private key without creating a different EOA address and moving assets. This makes key replacement operationally disruptive for people who maintain NFTs, signed orders, attestations, ENS identities and numerous token approvals. EIP-7702 improves delegated behavior for EOAs, but Ethereum warns that the original EOA private key retains ultimate account control; delegation alone does not remove this underlying authority [R04].

A smart account changes the primitive: the address is contract code and state, while authorization is replaceable. It has its own price: every new feature becomes additional attack surface in logic that may hold everything its owner possesses. Consequently KNOAD must resist the temptation to pack an unaudited framework of privileged modules into the core.

2.2 Credentials, addresses and cryptographic exposure

A private signing key determines its public key. Changing the public key alone while keeping the same ECDSA private key is not an available operation. A recovery phrase may stay fixed while derived or independent transaction key pairs change; meanwhile the wallet's smart-account address stays constant. An Ethereum address is a compact hash-derived identifier rather than the full public key. Normal ECDSA signatures allow recovery of the associated public key, which matters in future quantum threat models. The proposed onchain hot-signer **address commitment** aims to avoid revealing the replacement public key before its first authorized use, subject to the actual signature protocol and wallet implementation [R07].

2.3 Approvals survive key rotation

An ERC-20 allowance lives in a token contract. ERC-721 and ERC-1155 operator grants live in NFT contracts. Permit-style signatures may authorize external spenders without a separate `approve` transaction. Replacing a hot signer does not wipe those permissions. An approved spender can often execute transfers directly against the owner's assets without producing a new wallet signature. These controls must be handled independently from signer rotation. Key rotation and allowance management are **complementary**, not interchangeable.

2.4 AI and quantum threats—without exaggeration

AI-assisted vulnerability research and automation can reduce the cost of finding exploitable bugs. That is an application and protocol security problem; it does not imply that properly generated BIP-39 phrases can be guessed by modern AI. A 24-word mnemonic encodes 256 bits of entropy when correctly generated. Quantum computing presents a different, future threat to ECDSA and related primitives. Ethereum states no presently known quantum computer can break its cryptography, and it is developing cryptographic agility and post-quantum migration paths [R07]. NIST has already standardized post-quantum digital signatures such as ML-DSA [R08].

KNOAD must never claim that automatic key replacement alone is "quantum proof." An attacker able to recover a live private key during the window between signature exposure and effective signer retirement may still attack. A standing ECDSA recovery authority, if ever exercised without replacement, also remains a quantum-relevant control path. Defensive cryptographic agility is therefore a separate roadmap requirement.

2.5 The consumer problem

Security today commonly asks users to track approvals on external websites, understand the difference between transaction and message signatures, manage stuck nonces, manually revoke NFT operators, notice phishing interfaces, and protect a highly consequential backup phrase. KNOAD's product hypothesis is that **well-engineered defaults** can reduce that workload, provided the wallet is transparent when an action requires continuing privileges. "No friction" is an objective for ordinary operations, not a reason to suppress important security decisions.

03. Design Philosophy & Core Principles

3.1 Self-custody without hidden administrators

User-control means no Illuminary Labs account, API credential, validator, paymaster, recovery operator, pause switch or privileged upgrade key can unilaterally transfer or freeze user assets. A user must be able to independently construct and submit operations through ordinary public infrastructure. Where a feature requires server assistance, it must be optional and replaceable by another operator. Software supply-chain security is still important: an open-source specification is no protection if a compromised application tricks users into signing theft.

3.2 Minimal trusted core

The minimum privileged contract comprises (a) ERC-4337 validation, (b) strictly defined execution, (c) generation-bound hot signing, (d) separately authorized recovery and (e) a precisely scoped long-lived signature-validation mechanism. Permission-analysis and asset-indexing engines are preferably **client-side, informational or externally replaceable** and should not receive unlimited execution authority. Any contract module installation, upgrade or delegatecall facility is a possible bypass and needs an explicit authorization proof and separate audit.

3.3 Security objective hierarchy

When objectives conflict, prevent unauthorized spending first; preserve lawful owner recoverability second; maintain protocol correctness and reliable transaction execution third; protect long-lived orders and dApp compatibility fourth; minimize prompts and gas fifth. The hierarchy matters: a transaction should not secretly preserve an unlimited allowance merely for convenience, nor should an aggressive auto-revocation destroy the very NFT listing a user intentionally left active. User choices can change operation scope, but cannot waive fundamental contract authorization.

3.4 Stated non-goals

The wallet does not guarantee restoration after all recovery secrets are lost; it does not automatically migrate existing assets; it does not guarantee inclusion against chain-level censorship; it does not undo theft or exploitable marketplace state; it does not automatically remove every external approval; it does not give absolute quantum safety with ECDSA; and it does not promise every legacy dApp understands ERC-1271 smart-account signatures. These are boundaries of the architecture, not defects to conceal.

PART II — THE WALLET & ITS ARCHITECTURE

04. The KNOAD Wallet Experience

4.1 First launch and account creation

A user downloads an independently verifiable mobile app or extension, reviews a clear explanation of noncustodial risks, and chooses standard or advanced recovery. The app generates a 24-word recovery phrase locally using a cryptographically secure random source; optionally the user sets an independent BIP-39 passphrase. The application verifies the backup through a restoration exercise that does not transmit any secret. It derives a recovery authority and creates a fresh day-to-day signer. The account is deployed using a factory whose authorization and deterministic discovery rules have been tested against front-running and ownership substitution. Only after account initialization and recovery confirmation should the UI encourage depositing substantial assets.

Important distinction: the password used to open the app is not the recovery passphrase. The local password encrypts the daily signer vault. The mnemonic, with optional BIP-39 passphrase, is the user-held recovery secret. User choice must make the consequences of losing each one unmistakable [R06].

4.2 Daily activities

The home screen presents native balances, fungible tokens, NFTs, portfolio history and the active network. Sending funds, approving purchases and interacting with dApps should look familiar. For a typical onchain operation the app generates the next hot key, simulates the complete action, obtains the user's approval, signs with the current key and submits a UserOperation. Upon final, onchain observation of successful rotation, the client discards superseded hot-key material and transitions to the next authorization state. A status such as "submitted" cannot be conflated with "completed."

A receive operation does not reveal the current hot public key; it simply transfers assets to the stable account address. Tokens, contracts or identity systems that assume `tx.origin` denotes the wallet or require an EOA must be flagged as incompatible rather than silently handled through an unrelated account [R04].

4.3 Security Center

The Security Center offers **Refresh Signing Key**, **Review Permissions**, **Disconnect Site**, **Manage Active Orders**, **Recovery Setup** and **Transaction Activity**. It must not imply that disconnecting a website revokes an ERC-20 token allowance. Warnings distinguish a pending revoke transaction from one confirmed in token state. A user can select approvals for revocation or delegate a clearly bounded, confirmable batch that may fail partially under token-specific behavior. As an advanced option, risk monitoring can point out high-value unlimited approvals and unrecognized operators; it must accurately state chain coverage and the completeness of its indexer.

4.4 Existing wallets and interoperability

KNOAD will not need authority to relocate assets from MetaMask, Rabby or hardware wallets. Owners may migrate voluntarily using conventional transfers. ENS can be directed to the new stable smart-account address; identity continuity is straightforward **after** that initial move, not before it. Third-party dApps may need users to reconnect once on migration. WalletConnect and EIP-1193 interfaces should expose the smart-account address, while ERC-1271 validates supported offchain signatures. Where a site requires EOA-only verification, KNOAD must disclose incompatibility.

4.5 Accessibility and user language

The app should say “Your transaction is waiting” rather than “nonce lane stale,” but offer inspectable transaction IDs and account state to advanced users. In the recovery experience, it should say “Use your recovery phrase to replace the active signer” rather than “reset wallet password.” No sensitive secret may be pasted into a third-party site to diagnose a problem. Address book integrity, anti-address-poisoning display, network labels, clear transaction breakdowns and high-contrast signing screens are integral security controls.

05. Smart-Account Architecture

5.1 ERC-4337 components

ERC-4337 defines an alternate transaction flow in which clients submit a `UserOperation`, bundlers deliver operations to a trusted EntryPoint, the account validates its operation and then executes user-defined calldata [R01]. Its `nonce` can be partitioned into a 192-bit key and 64-bit sequence, permitting separate logical nonce lanes. The wallet MUST pin the EntryPoint interface and deployed address supported by each chain, verify protocol version compatibility and avoid unilateral reliance on a bundler. A bundler does not own user funds; it is a transaction service that can still censor or drop work.

The core modules are: `KnoadAccount` (asset-holding address and validation), `KnoadFactory` (deterministic deployment), `HotSignerPolicy` (single current signer and generation), `RecoveryPolicy` (separate authorization and replay state), `OrderAuthorizationPolicy` (scoped ERC-1271 verification), and `ExecutionPolicy` (bounded batching, call controls, revocation path). Their eventual combination may be modular or integrated, but **every route able to perform arbitrary calls must enforce the same security checks**. ERC-7579 offers a modular-account interface but remains a draft; use of modules expands the audit surface [R03].

5.2 Authorization and contract state

A representative logical state is: current signer identifier; monotonically increasing hot generation; recovery authority identifier and recovery epoch; locked-down recovery nonce lane; approved order commitments with verifier, expiry, status and security epoch; and basic replay/configuration metadata. The account should not keep users' mnemonic words, plaintext passwords, deterministic spending roots or recoverable private keys onchain. Public signer addresses are safe to publish against conventional preimage attacks but do not erase future quantum exposure from any signature already revealed.

The contract MUST reject calls from a non-EntryPoint caller except carefully specified self-calls or a separately authenticated recovery path. The account MUST reject `delegatecall` to untrusted implementations, self-authorization through alternate calldata, unauthorized module installation, reinitialization, and execution paths that omit rotation. All signature domains must be bound to the correct chain, account, EntryPoint, operation type, version and nonce as applicable.

5.3 Factory, deterministic addresses and account discovery

An address must be recoverable from stable information available to the owner even after device loss. The first research factory hashes the initial hot signer into the CREATE2 initialization bytecode. **That is a blocker** if hot keys are device-generated and ephemeral: the mnemonic alone cannot necessarily recreate the initial signer. Production candidates include a stable recovery-identifier-derived deployment salt and constant init code, coupled with factory-side initialization authorization, or an independently indexed event registry where the public recovery identifier locates accounts. Either choice MUST prevent factory front-running, malicious first initialization and account substitution, and MUST allow a user to prove the recovery credentials control the correct account before depositing assets.

The factory address, deployed bytecode hash, account version, recovery derivation scheme, chain ID, account index and discovery procedure must be documented in a portable recovery descriptor. No one proprietary API may be the sole locator. The developer must evaluate whether deriving/disclosing the initial recovery public

identifier causes unwanted exposure, especially if deployment requires a recovery signature. A deployment design that prematurely exposes the only standing recovery key would defeat a central security goal; one path is an initial recovery-key handoff, but that needs its own verification and costs.

5.4 Upgradability and implementation selection

A smart account able to delegate into arbitrary upgrade code is highly dangerous. Three approaches require comparative formal threat modeling: (A) immutable minimal core with non-privileged external services, (B) tightly constrained user-authorized migration to a preaudited implementation, and (C) upgradeable proxy with recovery-authorized timelock and audited allowlisting. The stated product preference is minimal immutable authority plus an explicit owner-controlled future migration path; no decision about general upgradeability is final until emergency-failure testing identifies which risk dominates. The “fixability” of a buggy contract should never become an Illuminary Labs admin backdoor.

06. Cryptography & Automatic Key Rotation

6.1 Exactly what rotates

The stable smart-account address does not change. The seed phrase need not change. The **authorized hot key pair** changes: both its public and private parts are replaced as a unit. A successful operation makes the old signing key cryptographically ineffective for new wallet transaction authorizations, subject to any separately, specifically granted signatures or pending operations whose state must be checked. Rotation is an onchain policy update, not the destruction of the old private key as a mathematical object.

6.2 Recommended key separation

The early prototype deterministically derived an entire hot-key sequence from the same 24-word seed. That is recoverable but introduces a concern: keeping the spending derivation root on the daily device could expose all future hot keys if the vault or process is compromised. A stronger recommended **candidate design** is to generate ordinary hot keys as fresh, locally protected random keys, retain only the current and prepared next key, and reserve a separately derived recovery authority for rebuilding control after device loss. The recovery phrase is not stored persistently in the online daily vault. This eliminates the requirement that hot keys be deterministically reproducible: if lost, the separate recovery credential installs a new random hot signer. It does not prevent a live-device attacker from spending with an accessible hot key.

A deterministic hot branch remains a fallback candidate for hardware compatibility or interoperability, but should not be selected merely because it simplifies coding. The choice affects multi-device signing, storage, backup requirements and wallet portability. Whichever is adopted, the production white paper and audit must pin a precise derivation and key lifecycle specification.

6.3 Preparation and retirement

At time T0 the account authorizes signer A at generation G. Before a user authorizes an operation the device generates signer B and securely verifies local possession; the contract is given an address/commitment for B, not a public signature from B. The operation includes the intended calls, generation G, the new signer commitment, replay parameters and fee policy. The account validates A's signature and generation. Upon successful outer operation execution it atomically switches authorized signer to B and increments generation. Subsequent operations require B and generation G+1. The app marks A retired only after chain confirmation. The prototype's public proof from signer B is explicitly removed from the production design; a local possession check cannot guarantee the device will never lose B, which is precisely why emergency recovery is independent.

6.4 The window that remains

The current signer A may expose its ECDSA public key in a pending UserOperation signature. It remains valid until execution and confirmation of its retirement. A sophisticated future quantum attack could exploit that transient interval if the attack becomes fast enough. A current thief holding A's private key may submit an operation and win a race before rotation; this is not solved by the contract's intent. Rotation reduces *persistence* of exposed authority; it is not full theft prevention and does not replace post-quantum signatures. Signing order digests also publishes signatures to the recipient, which requires separate policy.

6.5 Generation, nonce and synchronization rules

The generation counter is a security epoch, not an ERC-4337 nonce. The account validates both. Multiple operations can be simulated while A is still active; after the first confirms and replaces A, a stale operation must be rejected at execution even if a bundler previously validated it. The account must recheck generation immediately before any user-effectful execution. The client must never infer active-key state from its own local pending queue or from a bundler's "accepted" response. It queries the canonical account state and correlated EntryPoint receipts and waits for chain-specific finality policy.

07. Transaction Lifecycle & Reliability

7.1 A successful normal operation

The normal path is **prepare next key → simulate full batch → user approval → sign with A → submit UserOperation → EntryPoint validation → account execution → rotate to B → confirm receipt and state → activate B locally**. The operation must be bounded in size and gas to prevent a malicious target from monopolizing execution and starving signer retirement. The wallet should display separate statuses for **requested business action** and **key rotation** because partial application failure can still lead to a valid account-state update.

A production implementation can place the user action within a rollback boundary and rotate the signer outside it. If the user action reverts, the outer account operation may still succeed and retire A while reporting the action failure in an event. This requires explicit consent and clear UI messaging: the user has paid gas but their intended asset action did not complete. If the entire account/EntryPoint operation reverts or runs out of gas, that outer rotation must not be reported as successful. Real behavior depends on the exact EntryPoint and account implementation and must be verified against actual execution traces.

7.2 Pending and replacement operations

A submitted operation does not immediately retire A. If the bundler drops the operation, A remains current and its relevant nonce sequence may be unconsumed. The client can resubmit or replace the pending operation at the **same ERC-4337 account nonce** under bundler-specific replacement rules, with an appropriate fee change. “Same nonce, higher gas” is a helpful EOA analogy, not a guarantee: a UserOperation involves bundler mempools, EntryPoint nonces, validation rules and fee requirements. A cancellation is a replacement operation designed to perform no user asset action; it can rotate the key if the outer operation completes. It is never safe to run two incompatible replacements with different next keys and assume the client will know which wins.

7.3 Failure classification

Condition	Onchain signer	Nonce implication	Client response
UserOp accepted by bundler, not included	A	May be unconsumed	Inspect all bundlers; prepare replacement or wait
Original operation included	B if rotation succeeded	Sequence consumed	Verify event + storage; activate B
Replacement wins	B (as encoded in winner)	Same sequence consumed	Abandon original; reconcile action effect
Inner dApp call reverted, outer account succeeded	B	Sequence consumed	Report app failure and successful rotation separately
Outer account execution reverted	Usually A; verify	Sequence may still be consumed	Re-query state and obtain current nonce
Transaction included then reorganized	Depends on canonical chain	May roll back	Reconstruct from finalized chain and retry safely
Network censorship or bundler outage	No guaranteed change	Uncertain	Try independent infrastructure; do not claim guaranteed liveness

7.4 Concurrency and two-device realities

A user's phone and desktop cannot each assume A remains live and independently prepare A→B and A→C. A single active-signing queue per account is the simplest safe policy at launch. Clients discover and reconcile pending operations by chain/account/nonce/generation; when offline devices reconnect they must discard invalid generations. A second device without the hot key uses user-authorized device handoff or the independent recovery path. Multi-device shared signing roots should be considered only after an explicit threat model; silent cloud synchronization of a root secret would conflict with our strongest isolation principle.

7.5 Gas, anti-grief and failure bounds

The operation needs enough gas to validate, execute the bounded user action, write rotation state and emit status. Malicious tokens and marketplaces can revert, use substantial gas, reenter callbacks, or return huge error payloads. Contracts must cap call count, returndata handling and dynamic decoding; use the smallest necessary external-call surface; and prove there is always a recoverable state after any revert. Recovery must have its own replay counter, role and nonce lane so a stuck normal operation is not the only path to new authority. Emergency recovery is not guaranteed to be included during chain-wide censorship or insufficient gas availability.

08. Recovery & Ownership Continuity

8.1 Ownership and daily authority are different

Everyday hot keys provide spending authority under a deliberately temporary policy. The 24-word user-owned phrase derives a separate recovery authority able to replace the current hot signer without relying on it. The recovery authority cannot be changed by an ordinary hot-signing operation. In no case can KNOAD reset that recovery authority, access the user's seed, or approve a recovery request on behalf of the user. A permanent public recovery identifier is an authorization verifier, not a stored password.

8.2 Recovery material and optional enhanced secret

For standard users, a properly generated 24-word BIP-39 mnemonic is the offline recovery foundation. An app unlock password encrypts the local hot-key vault and is not part of the recovery protocol. Advanced users may enable a separate BIP-39 passphrase that combines with the mnemonic to derive **both the recovery root and any mnemonic-derived credentials**; using it for recovery alone while leaving spending derived from the unprotected phrase can create misleading protection against mnemonic theft. The BIP-39 standard specifies PBKDF2-HMAC-SHA512 with 2,048 rounds; a weak human-chosen passphrase can still be guessed offline if an attacker obtains the words and a verification target [R06]. A high-entropy passphrase strengthens theft resistance but adds another irrecoverable failure mode if forgotten.

This paper recommends **standard recovery by default, strong optional enhanced mode**, with conspicuous warnings and a verified two-secret restoration ceremony. Neither mode recovers an account if all valid recovery material and usable hot keys are irretrievably lost. That is a property of sovereign custody, not a service outage.

8.3 Emergency refresh procedure

On a fresh device, the user enters the phrase locally (plus optional BIP-39 passphrase). The app discovers the account through the documented factory/discovery procedure; checks the current recovery epoch, active signer and nonces; derives the correct recovery credential; creates a new random hot key; prepares a typed, tightly scoped recovery operation including account, chain ID, new hot signer, recovery counter, deadline and optionally a rotated next recovery identifier; signs locally; simulates; and submits via any compatible bundler. The account validates the recovery authority on its independent lane, installs the new hot signer and invalidates outstanding prior-generation hot authorizations.

The recovery UI MUST show the exact destination account and the action being authorized. Recovery never needs permission from the current hot signer, and no cloud request is a cryptographic prerequisite. A user who still has the current hot signer can perform an ordinary manual refresh without entering the phrase.

8.4 Recovery authority durability and rotation

A standing ECDSA recovery address is a sensitive long-lived authorization path. An emergency recovery signature reveals material relevant to future quantum attacks, and an already compromised recovery key could seize control again unless rotated or permanently retired. KNOAD should treat recovery as a **one-time recovery epoch**: successful use installs a fresh recovery authority derived from a documented next index or another explicitly authorized scheme and rejects all old recovery signatures. The design must prove a legitimate user can discover the current recovery index without a KNOAD server, can verify the next authority before installation, and cannot be locked out by a malformed next index. Recovery "refresh" requires both cryptographic proof and a functioning onchain verifier, not merely knowing the wallet address or supplying the app password.

8.5 Adversarial recovery and theft race

If only the hot key is stolen, its owner may race to drain assets or attempt unauthorized rotation, but it MUST NOT be able to edit the recovery authority. A valid recovery operation should supersede the hot generation and invalidates pending hot actions upon execution. If the complete recovery phrase is stolen in standard mode, the attacker may reconstruct recovery authority and seize assets; the chain cannot tell the original holder apart from the thief. With advanced BIP-39 passphrase mode, theft of the words alone need not suffice, but a compromised daily device can still lose currently spendable assets. Recovery cannot claw back already settled transfers.

8.6 Fresh-device proof and independence

Before any release, a clean installation must locate an account with only its published recovery materials and a selectable RPC plus alternative bundler; it must correctly identify the live recovery epoch, avoid misbinding to a stranger's account, and replace an inaccessible hot key. The client should publish an open-source, minimal recovery command-line tool and test fixtures so an owner can recover without the KNOAD application, web domain or company database. Reproducibility of this exercise is a release gate.

09. Proactive Permission Management

9.1 Three different classes of access

The wallet must distinguish **transaction spending authorization** (current hot signer), **onchain token/operator allowances** (token contract state), and **offchain signed authorizations** (permit, orders, logins or user-defined delegations). They have different owners, lifetimes and revocation mechanics. A client that says “keys rotated, all permissions revoked” would be wrong. Disconnecting a site usually only ends a dApp session and does not itself change token approvals or revoke a marketplace order.

9.2 Least authority at the start

Before asking a token contract to `approve`, KNOAD should inspect the exact spender, token, amount and desired action. It should prefer exact-amount approvals, `permit` or protocol-native single-use authorizations where safely supported, and atomic **approve → execute → revoke** batches if the complete workflow can be processed within one owner-authorized account operation. Token-specific failure behaviors, contracts that do not return booleans, allowance update quirks and fee-on-transfer tokens must be handled by vetted adapters rather than by optimistic assumptions.

When a transaction intentionally depends on standing approval—for instance a pending Seaport listing or an external vault—the wallet must preserve it after clear user consent. It can mark such grants as “standing for active order,” monitor expiry and offer cancellation, but cannot safely remove them unilaterally without potentially breaking the order. If an allowance is consumed completely, the risk may end naturally, yet the wallet should still verify actual remaining state.

9.3 Monitoring is not perfect knowledge

An app may index ERC-20, ERC-721 and ERC-1155 events and contract state, query allowances for known spenders, detect approvals it submitted and infer risky permissions. It cannot necessarily enumerate every spender in every ERC-20 contract, since the ERC-20 interface provides no universal allowance enumeration. Custom permit systems and offchain commitments may not appear onchain until used. The Security Center therefore MUST disclose indexed chains, tokens, supported protocols, data freshness and blind spots. “No known risks found” is not equivalent to “no permissions exist.”

9.4 Revocation semantics

Revocation is an onchain call where the underlying contract supports it: `approve(spender,0)` or a protocol-specific replacement for ERC-20; `approve(address(0), tokenId)` or `setApprovalForAll(operator,false)` as applicable for ERC-721; and `setApprovalForAll(operator,false)` for ERC-1155. Some permit-based signatures need nonce invalidation in their own system. Revoke operations cost gas, can fail or be blocked, and cannot reverse an already executed transfer. “Auto cleanup” should mean: automatically propose or include bounded cleanup when demonstrably safe, not background seizure of arbitrary rights.

9.5 Consent and action simulation

Every approval screen should identify who can spend, what can be spent, the maximum scope, a timeout if enforced, and whether the grant survives key rotation. The wallet must distinguish a site domain from the actual onchain spender address and show verified addresses only where strong verification exists. Simulation is a tool,

not a proof: onchain state can change between simulation and execution, and malicious contracts can branch on block context. Blocking dangerous broad approvals by default is useful, but the exact reject/override policy remains a product and security review decision.

10. Persistent Signatures & Delayed Transactions

10.1 Why rotation threatens listings

An NFT listing or buy offer is often a signed offchain order. A marketplace keeps the order and later fulfills it after a buyer or seller appears. If `isValidSignature` only recognizes the **current** hot key, an otherwise legitimate order becomes invalid when that key rotates. That would make everyday wallet activity break ongoing sales and offers. Conversely, allowing every signature from every retired key to remain valid would defeat the purpose of rotation and permit broad replay. Both extremes are unacceptable.

10.2 Exact, revocable signature grants

ERC-1271 lets the account decide whether a signature is valid for a particular hash [R02]. The proposed approach is a scoped order-grant registry containing the **exact signed digest**, specific accepted verifier or protocol scope, explicit expiration, signer proof/authorization at grant time and a revocation state. A retired key is never treated as a continuing general signer. Its signed order can remain valid **only for that registered digest under the intended context**. Emergency refresh can advance a security epoch to invalidate grants deemed unsafe; routine rotation leaves them intact by default.

A key design risk is `msg.sender` within `isValidSignature`: exchanges may call through wrappers, conduits or universal verification contracts, so a simplistic requirement that the direct caller equal the apparent marketplace might reject legitimate orders. More importantly, signature hashes reused across accounts can introduce replay vulnerabilities. ERC-7739 proposes defensive rehashing of ERC-1271 signatures, and implementation-specific domain separation must be audited for each signing route [R09].

10.3 Seaport and order execution dependencies

OpenSea's Seaport documentation describes ERC-1271 order signature validation [R10]. But Seaport support for contract signatures does not ensure every wallet UX or bulk-order packing scheme will accept a proprietary signature wrapper. Single-order signing, exact EIP-712 digest computation, verification, marketplace posting, fulfillments, bulk orders, cancellations, partial fills, conduits and counters require end-to-end tests. The account must not invent an order digest independent of what Seaport verifies.

Even valid orders can be unfillable if token ownership changes, token approvals are revoked, a counter changes, a deadline passes, funds become insufficient, or a conduit is no longer approved. A buy offer funded in WETH may depend on a persistent allowance and balance. The wallet should label those dependencies and never promise that keeping an ERC-1271 signature valid ensures eventual fulfillment.

10.4 Economically acceptable authorizations

Onchain registration of an individual order hash adds gas to what some marketplaces otherwise treat as an offchain signature. Candidate optimizations include batched grants, short-lived scoped authorizations and well-reviewed signed-policy roots with nonces, limits and revocation; each increases complexity and demands separate adversarial testing. V1 should prioritize a secure, single-order path over a gas-optimal scheme. Any adapter must define what a stale order, cancellation, noncanonical marketplace implementation and emergency-recovery invalidation mean before it is enabled by default.

PART III — USABILITY, INDEPENDENCE & ADOPTION

11. Application Design & Web3 Integration

11.1 Client boundary and platform components

The client is the user's security-critical signing terminal. Android and iOS applications and desktop browser extensions have distinct attack surfaces, keystores, deep-link flows and process-lifecycle constraints. No website JavaScript context may read wallet secrets or import a mnemonic. The client should isolate a dedicated signing service from the dApp browser, route all requests through a permissioned RPC bridge, and bind every request to a verified origin and a displayed account/chain. The wallet connects a dApp through standard provider methods and WalletConnect rather than by granting a webpage access to its private cryptographic vault.

WalletConnect sessions are authorizations to communicate; they must be inspectable and revocable, but revoking the session must never be depicted as revoking onchain token approvals. Where a dApp asks for personal-sign or structured-data signatures, the wallet should render the full message or decoded typed structure and identify hidden permit/transfer/approval consequences. "Sign to log in" can carry significant financial risk if the site actually requests a spender authorization.

11.2 Integrated browser

The mobile browser is convenience infrastructure, not a privileged actor. It needs strict origin isolation, navigation indicators, URL anti-homograph handling, appropriate sandboxing, safe external-link handling, and a clear transition into a trusted wallet signing sheet outside the webpage's DOM. Injection of an EIP-1193 provider should be limited to appropriate contexts and privacy-conscious permissions. The wallet should refuse to show a misleading "verified" badge for merely having an HTTPS certificate. All updates to the signing host, webview engine and dependencies require supply-chain monitoring.

11.3 Local secret storage

For a password-protected software vault, generate a random vault-encryption key and protect it with an appropriately configured Argon2id-derived wrapping key or platform-authenticated keystore, then use authenticated encryption such as AES-GCM with unique nonces and authenticated metadata. OWASP recommends memory-hard schemes such as Argon2id for password-derived key protection [R11]. A password **hash** is not a usable symmetric encryption key by itself without a properly specified derivation and encryption design; MD5 must not be used. The salt and cipher parameters may be stored in the encrypted vault header; the phrase and optional BIP-39 passphrase are not stored in the onchain account.

Where device hardware can protect a particular signing algorithm, use it only after verifying the key cannot be exported, the algorithm actually matches the account validator and backup behavior is understood. Popular platform hardware can differ in whether native secp256k1 signing is supported; software fallbacks must be explicit and audited. A biometric unlock is an activation mechanism, not a blockchain recovery secret. Recoverability after device destruction must not depend on that biometric remaining available.

11.4 Account state as the source of truth

The app must maintain a local cache for performance without treating it as authoritative. On reconnect it reads verified account state, chain ID, contract code/hash, signer generation, recovery epoch, known outstanding UserOperation hashes and EntryPoint nonce lanes. Portfolio and transaction-history indexers are replaceable,

and balances can be validated through direct RPC reads. The app should handle malicious RPC responses, reorgs, false token metadata, scam NFTs, unexpected decimals and chain IDs that differ from displayed network names. Account state must never silently roll backward because a local cache is stale.

11.5 Compatibility principles

Supported assets in V1 include native currency, ERC-20, ERC-721 and ERC-1155. The interface should also understand atypical token behavior well enough to identify failures, not assert universally correct transfer simulation for every nonstandard token. DApps requiring an EOA signature cannot be made compatible by misrepresenting a contract account as an EOA. ERC-1271 login and signed order compatibility must be tested against real dApps. ENS name resolution points at the stable smart account, and addresses shown for incoming funds must remain correct even as authority rotates.

11.6 UX acceptance tests

Before release, user research must verify that ordinary holders can distinguish send from approve, view pending state, cancel when permitted, recognize a persistent marketplace order, perform manual refresh, back up recovery credentials, and restore on a clean device **without** having access to company infrastructure. The product must include small-screen layouts, screen-reader labels, clear signing screens, accessibility fonts and meaningful error recovery. Zero-intervention defaults should never require a user to accept unchecked broad permissions.

12. Decentralization & Infrastructure Independence

12.1 A precise definition of decentralization

For this product, noncustodial means that users, and only parties with credentials the users have cryptographically authorized, can cause the account to spend or change its control. Decentralized usability means independently deployable/verifiable contracts, an openly documented account factory, and multiple submission and discovery providers—not a claim that every mobile operating system, app store, RPC provider or blockchain is decentralized. The account should function even if knoad.io, Illuminary Labs and every KNOAD-hosted API go offline.

12.2 Independent operation procedures

A user must be able to install an alternate client implementing the public format, derive or provide their recovery authority, locate the account using documented factory descriptors or an alternate index, read account state, construct a compliant UserOperation and submit through an independent bundler. A public test fixture should provide example recovery derivations, expected addresses, generation transitions, recovery epoch updates and signature vectors that any implementation can reproduce. Where the chain's EntryPoint is the account's operational dependency, that dependency and the chain's upgrade policies must be explicit.

KNOAD must not bake an unreplaceable API URL, opaque signature coordinator, whitelist-only relayer or secret corporate signer into validation. Third-party paymasters may offer a convenience service, but the owner must retain an ordinary user-funded path whenever the chain permits it. The company can discontinue front-end development, yet user control must persist under published contracts and recovery tools.

12.3 Contract upgrades and rescue tradeoffs

True permissionless ownership creates a hard constraint: there is no corporate "fix everything" button if a contract is fundamentally broken. A general public upgrade mechanism increases the chance of an owner losing funds to malicious implementation or authorization bugs. An immutable core reduces administrative attack surface but locks in design mistakes. A narrow, independently recovery-authorized migration procedure—only to user-reviewed destinations and never using privileged operator signatures—could provide bounded flexibility, but it still requires proving that the old contract remains capable of authorizing the migration. The final account deployment strategy needs external assessment of these alternatives and must be transparent about what a total contract failure could make impossible.

12.4 Generational ownership and estate planning

"An asset can be inherited" is not equivalent to having a safe inheritance process. KNOAD can document user-controlled backup and transfer options, but must not insert mandatory guardians, custodial dead-man switches or privileged company executors by default. An advanced inheritance module may eventually support owner-approved delayed transfers or multiple personal keys, but it creates its own dispute, privacy, censorship and attack surfaces. V1 should keep the recovery mechanism minimal and make the limits of user-owned backup explicit.

PART IV — SECURITY, VERIFICATION & TRUST

13. Security Architecture & Threat Modeling

13.1 Assets, principals and capabilities

Protected assets include onchain balances, NFT ownership, signed orders, token allowances, identity entitlements, offchain authorization signatures, active hot private keys, device vault contents and recovery secrets. Principals include owner, local operating system, wallet application, hostile dApp, token contract, marketplace, RPC provider, bundler, EntryPoint and recovery client. High-risk trust boundaries occur when an untrusted website asks for a signature, when a UserOperation moves from client to bundler to EntryPoint, when the account calls a token, and when recovery is requested from a fresh device.

13.2 Non-negotiable authorization invariants

The production contract MUST enforce at least the following invariants, subject to formal statements pinned to actual code:

1. **No arbitrary spend without current authority.** Every entry point capable of moving assets must validate an authorized hot or recovery signature in the correct role and domain.
2. **At most one current hot signer.** Completing a valid rotation invalidates the prior hot signer for all future routine operations.
3. **Generation binding.** No stale generation executes a user-effectful call after an intervening rotation or recovery.
4. **Recovery independence.** A hot signer cannot install, disable or redefine the recovery authority, directly or by module/upgrade path.
5. **Recovery supersession.** A valid recovery changes the hot generation and renders prior pending hot operations ineffective on the canonical state.
6. **No credential-free key change.** Knowledge of an address, possession of an app password, or connection to a website cannot authorize any onchain signer refresh.
7. **Exact-order authorization.** A retired key may validate only explicitly registered, bounded digests; no wildcard message signing survives retirement.
8. **Replay separation.** Signatures are bound to account, chain, nonce/epoch, action type and allowed lifetime, with appropriate handling of smart-account typed signatures.
9. **Rollback accuracy.** The UI and event model never mark a spend successful when only outer key rotation succeeded, nor mark rotation successful when outer execution reverted.
10. **Vendor independence.** A recovery operation can be reconstructed and submitted without a KNOAD service or private corporate signature.
11. **No unauthorized account initialization.** Factory and initialization flows cannot be claimed or modified by a racing adversary.
12. **No automatic open-order destruction.** Routine rotation alone does not invalidate an explicitly authorized ongoing order, unless a declared user policy requires it.
13. **No concealed standing grants.** The wallet may not describe persisted token allowances as revoked merely because the signing key changed.

14. **No hidden root exposure.** Daily key-rotation mechanisms never require disclosing a recovery seed or recovery passphrase to the dApp, RPC, bundler or backend.

13.3 Threat catalog

ID	Adversary / failure	Expected impact	Primary control	Residual risk
T01	Theft of active hot key	Unauthorized spend before retirement	Rapid rotation, local isolation, simulation and warnings	Race-to-execute remains
T02	Theft of 24 words	Unauthorized recovery in standard mode	Offline secret; optional strong passphrase	No cryptographic attribution of “real” owner
T03	Rogue signer rotation	Permanent hot-signing loss	Independent recovery, no hot control of recovery policy	Already-settled theft unrecoverable
T04	Unreachable replacement key	Owner locked out of hot path	Fresh locally tested key + owner recovery	Recovery implementation failure
T05	Stale nonce/operation	Wrong key shown or unavailable action	Reconcile canonical state, recovery lane	Bundler censorship and chain liveness
T06	Malicious token/ spender	Drains via standing approval	Exact grants, bounded cleanup, risk center	Unenumerated permissions; adversarial protocols
T07	Invalid long-lived orders	Broken listings and offers	Scoped ERC-1271 authorization	Marketplace-specific incompatibility
T08	ERC-1271 replay	Unauthorized signature acceptance	Domain separation, ERC-7739 review	Complex third-party verification paths
T09	Upgrade/module takeover	Arbitrary wallet execution	Minimal core, narrow vetted upgrade path	Contract logic vulnerabilities
T10	Factory front-running	Account hijack before deployment	Authenticated initialization, deterministic discovery	Deployment design not finalized
T11	Device malware	Live-signing theft or UI deception	Isolated signing, secure vault, transaction display	Compromised device may still sign theft
T12	Quantum breakthrough	Recovery of ECDSA keys from public signatures	Shorter authorization duration, crypto agility	Not PQ-secure until genuine PQ validators
T13	RPC/bundler outage	No operation included	Multiple providers, independent tooling	Chain-wide censorship can persist
T14	Contract fatal bug	Assets inaccessible despite phrase	Audits, minimal code, independent remediation study	No guaranteed universal rescue

13.4 Compromised device and browser threats

Wallet security is not just contract security. A malicious application update, injected extension, deep-link replacement, clipboard hijack or hostile dApp bridge may redirect transfers while the contract correctly verifies the authorized signer. The client should reconstruct calldata from a typed user intent, display the actual destination/amount/approvals it will sign, protect the signer process from webpage code, sign narrowly scoped typed messages and maintain software supply-chain controls. Device compromise can still defeat software-only signing, so the paper does not promise that the wallet prevents all asset theft.

13.5 Quantum threat boundary

In today's ECDSA implementation, public-key revelation can occur when an authorized signature becomes available, not only when a transaction confirms. Post-transaction rotation lowers the period during which that particular key remains useful as an account controller. It cannot stop an attacker that compromises the active key quickly enough, a stolen recovery root, an evergreen externally granted allowance, or a future chain-level cryptographic failure. Long-lived orders registered through ECDSA proofs and recovery actions must also be reviewed against quantum exposure. Post-quantum transition will require an audited supported signature validator and protected migration of **all critical authority paths**, including recovery and signatures for existing authorizations [R07][R08].

13.6 Risk acceptance and transparency

There is no honest "all funds always safe" guarantee. The team must publish remaining risks and severity criteria, including the irreversible consequences of missing recovery materials, malicious approvals, account-contract bugs and delegated third-party actions. A bug bounty and a coordinated disclosure program should cover both Solidity and secret-handling clients. Public risk statements should evolve with actual external audits; an unaudited design must never be advertised as audited by implication.

14. Engineering Standards, Testing & Auditing

14.1 State of the earlier proof of concept

The October 13 research archive contains a Solidity rotating-account draft, a CREATE2 factory, a helper app, JavaScript state-model tests and a preliminary review. It is not a production implementation and cannot be cited as having passed Foundry, independent review or marketplace certification. It uses a next-key proof signature, lacks the finalized recovery path, binds CREATE2 address to the initial hot signer, and has an order-verifier design whose behavior with real Seaport contracts must be established. These are not cosmetic issues. The next development phase begins from an audited account core and replaces, rather than merely patches around, unsafe assumptions. See Appendix J for specific prototype disposition.

14.2 Verification layers

Layer one: mathematical/model requirements. State-machine invariants define legal transitions among hot generations, recovery epochs, nonces and order commitments. Model-check or property-test all transitions, including unexpected reverts and rollbacks.

Layer two: Solidity execution. Compile against pinned dependencies and a real EntryPoint version; run unit tests, differential tests, fuzz tests, invariant/property tests, gas snapshots, static analysis and adversarial harnesses. No handwritten mock is sufficient to establish ERC-4337 validation/execution semantics.

Layer three: client cryptography. Audit mnemonic generation, derivation, local vault encryption, random number generation, secure key deletion, signing format, chain binding, wallet browser boundaries, dependency integrity and fresh-device recovery. Include Android and iOS device variants and failure-injection tests.

Layer four: ecosystem interoperability. Exercise wallet connections, ENS, standard token operations, Seaport listings and offers, login signatures, Permit2 interactions, cancellation, revocations, major bundlers and independent RPCs on every claimed network.

Layer five: external review. Commission independent specialized smart-account, cryptographic implementation and wallet-client audits. Publish scope, commit hashes, findings, fix commits and retest results. External review is not a substitute for continued monitoring.

14.3 Failure-injection matrix

Test class	Minimum test	Passing requirement
A01	Wrong current signer	Rejected with no asset movement
A02	Wrong next key/address	Controlled failure with valid recovery still possible
A03	Reuse retired signer	No spend/upgrade/recovery-policy change
A04	Two pending A-generation operations	At most the valid winner can execute effects
A05	Same-nonce fee replacement	Canonical winner reconciled; no ghost success

Test class	Minimum test	Passing requirement
A06	Inner dApp revert	Action outcome and rotation outcome correctly separated
A07	Outer out-of-gas/revert	No false rotation; owner can retry or recover
A08	Reorg after rotation	Client rolls back provisional signer state safely
A09	Lost device and hot key	Mnemonic-only standard recovery succeeds offline from KNOAD
A10	Advanced passphrase absent/wrong	Wrong passphrase cannot recover intended account
A11	Compromised hot signer edits recovery	Always rejected on every code path
A12	Recovery replay	Invalid after recovery epoch change
A13	Factory account front-run	Attacker cannot bind chosen account to wrong authority
A14	ERC-1271 order after rotation	Only exact intended digest remains valid
A15	ERC-1271 cross-account replay	Rejected under binding policy
A16	Revoked NFT approval	Token contract reports permission removed
A17	Standing order approval	Preserved only per user-acknowledged policy
A18	Malicious token callbacks	No reentrancy or role bypass
A19	KNOAD backend unavailable	Owner submits and recovers via independent stack
A20	Adversarial client origin	Untrusted webpage cannot retrieve signer/seed

14.4 Audit deliverables and release gate

Prior to any meaningful asset-backed pilot: frozen threat model; implementation-specific spec; independently reproduced deterministic address and recovery vectors; all mandatory security invariants passing; independent audits of contract and app; all critical/high severity findings remediated and retested; well-understood medium-risk deviations documented with owner consent; reliable alternate-bundler recovery; reproducible deployed bytecode; and a staged rollback/communications plan. A pilot must use strict asset caps, intentionally limited participation and an incident escalation path. The organization should not publish “secure”, “quantum-proof” or “fully audited” until evidence matches those phrases.

PART V — SUSTAINABILITY & THE FUTURE

15. Economics & Operational Sustainability

15.1 Free does not mean gasless

Core wallet software, hot rotation, the Security Center and owner recovery should be free of KNOAD subscription charges. Onchain actions still consume gas. ERC-4337 may involve extra validation, state writes and bundler economics compared with a basic EOA transaction. Order-grant registration and revocation can create additional cost. A paymaster may optionally sponsor bounded operations, but the account must retain a self-funded route. The business should disclose service fees, exchange spreads and RPC dependency costs rather than hiding them in optimistic “no-fee” language.

15.2 Gas measurement methodology

No fabricated gas price belongs in the specification. The team must benchmark create account, plain transfer plus rotation, ERC-20 swap with bounded approval cleanup, NFT listing grant, order revocation, normal refresh and emergency recovery under pinned chain and gas conditions. Measurements should include verification gas, preverification gas, call gas, EntryPoint overhead and any paymaster cost. Both median and worst-case realistic scenarios matter. Document the contract commit, EntryPoint, chain height, client, bundler and configured limits so results are reproducible. The price of a safer default must be evaluated against consumers’ tolerance for extra transaction fees.

15.3 Company operating model

Initial ongoing expenses include release engineering, security reviews, RPC/bundler access, asset-indexing infrastructure, simulations, monitoring, support and incident response. Nothing in the core authorization path should require a proprietary account server. Future monetization may include optional clearly priced swap/bridge interfaces, advanced noncustodial organizational features or sponsorship partnerships. Revenue services must not receive secret custody or authority to alter user balances. A wallet built for safety should not make broad approvals a hidden revenue incentive.

15.4 Privacy and data handling

Portfolio indexing may reveal addresses, app usage and transaction history. KNOAD should minimize telemetry, make analytics optional where practicable, document retention periods, separate crash reports from wallet addresses, and never collect recovery phrases or BIP-39 passphrases. Security alerts should prefer locally evaluated data when possible. If servers are used for opt-in notifications, the privacy/trust disclosure must explain what wallet metadata they see and what happens when the service disappears.

16. Development Roadmap & Future Capabilities

16.1 Phase I — Security architecture freeze

Select a supported audited ERC-4337 base implementation, pin dependencies and EntryPoint versions, finalize factory/discovery, and publish a complete technical specification. Produce recovery and hot-signing test vectors, choose a robust nonce and generation binding design, formally model recovery protection and decide whether modules/upgrades are allowed at all. The deliverable is a specification with no unresolved P0 access-control or account-discovery design gaps—not an app-store release.

16.2 Phase II — Contract and recovery implementation

Implement signer rotation with no premature incoming-key public signature, independently authorized and ratcheting emergency recovery, bounded batch execution, exact-digest ERC-1271 controls, replay separation and deployment authorization. Run local Foundry tests against real EntryPoint and deliberately hostile tokens. Prove that a clean second device can restore with only the recovery materials without KNOAD infrastructure. Publish an initial threat model and test matrix.

16.3 Phase III — Security-centered wallet clients

Build the encrypted vault, Android/iOS signing isolation, browser provider bridge, desktop extension, transaction simulation, active-order management, permission inventory and multi-RPC/bundler handling. Integrate major token types, WalletConnect and initial Seaport test cases. Test accessibility and failure messages with real users. Maintain independent recovery tooling alongside the main application rather than treating it as an afterthought.

16.4 Phase IV — Verification and staged rollout

Commission at least independent contract and client security reviews, remediate and retest findings, run long-lived adversarial testnets and measure gas economics. Restrict initial production deployment to a well-characterized chain, minimal permitted asset value and a small opt-in cohort. No security promise should outrun verified chain/platform compatibility.

16.5 Phase V — Expansion and cryptographic agility

Broaden chain support only after the same audit, deterministic discovery, recovery and marketplace requirements are satisfied for each chain. Study audited future digital-signature validators based on standards such as NIST ML-DSA or SLH-DSA and Ethereum's evolving post-quantum account infrastructure [R07][R08]. Transition plans must cover both routine and recovery control, historical persistent signatures, gas costs and failure-recovery procedures. Future organizational accounts, corporate policies and inheritance should remain optional user-chosen modules whose rights are explicit and cannot undermine base self-custody.

16.6 Roadmap gates, not dates

Delivery timing will depend on independent audit outcomes and protocol readiness. A calendar does not substitute for a proof. KNOAD's public roadmap should state *criteria to advance* each phase, along with accepted technical debt and rollback conditions. A requirement that cannot be implemented safely is deferred or removed; it is not marketed as a coming attraction while users hold funds under a weaker security claim.

17. Conclusion — The Future of Digital Ownership

A decentralized wallet should preserve user sovereignty across devices, institutions and generations. That ideal is undermined when every security improvement forces an owner to move assets, rebuild identity and repair their digital life. KNOAD Wallet's architectural thesis is that account identity can remain stable while authorization evolves: a temporary hot signer authorizes activity, the account enforces its retirement, an independent owner-held recovery authority restores control when necessary, and least-privilege permissions reduce avoidable standing exposure.

That thesis imposes responsibility. A smart account introduces new bugs that an EOA does not have. A phrase that is sufficient for recovery is also sufficient for theft when stolen. A marketplace order that survives rotation requires specifically preserved authority. An incoming signer can be unreachable; a bundler can censor transactions; an external token can retain a dangerous allowance. These realities must be explicitly designed, tested and communicated—not smoothed over in a promise that the user will never need to think about security again.

The intended achievement is therefore more precise than absolute safety: **make ordinary ownership familiar, keep critical control entirely in the user's hands, reduce the lifetime of routine signing authority, minimize unnecessary permissions, and prove that recovery does not depend on KNOAD's survival.** Where those properties can be formally or independently verified, publish the evidence. Where they cannot, document the limitation and delay the launch. Permanent ownership deserves no less.

TECHNICAL APPENDICES

Appendix A — Glossary and Terminology

Externally owned account (EOA): An Ethereum account traditionally controlled directly by an ECDSA private key. **Smart account:** A contract holding assets and enforcing programmatic authorization rules. **Hot signer:** The currently allowed short-lived key controlling ordinary wallet actions. **Recovery authority:** A separate user-held cryptographic signer capable of installing a new hot signer. **Recovery phrase / mnemonic:** Human-readable BIP-39 backup used to derive a cryptographic seed. **BIP-39 passphrase:** An optional independent input to mnemonic-to-seed generation, distinct from app unlock password. **App password:** A local secret used to protect the encrypted device vault. **UserOperation:** ERC-4337 object processed through EntryPoint. **Bundler:** Infrastructure actor that combines and submits UserOperations. **Paymaster:** Contract that can sponsor UserOperation gas under a policy. **Nonce lane:** ERC-4337 key+sequence logical nonce channel. **Generation / epoch:** Monotonic counter invalidating stale hot or recovery authority. **ERC-1271:** Method for contract accounts to validate offchain signatures. **Allowance:** Spending permission stored in an ERC-20 token contract. **Operator approval:** NFT-wide permission held by an operator address. **Exact-digest grant:** A specifically authorized signature hash that may remain valid despite hot signer retirement. **Liveness:** Ability to process authorized operations when the underlying chain cooperates. **Safety:** Prevention of unauthorized state transitions. **Quantum resistant:** Cryptographic construction believed secure against relevant quantum attacks, not simply a rotated ECDSA key.

Appendix B — Relevant Standards and Status

Standard	Purpose and treatment in KNOAD
ERC-4337 [R01]	EntryPoint/UserOperation account abstraction; production-critical.
ERC-1271 [R02]	Contract-signature verification for orders and logins.
ERC-7579 [R03]	Modular smart-account interoperability; **draft** and optional.
EIP-7702 [R04]	EOA delegation; not sufficient to remove original EOA key authority.
BIP-39 [R06]	Recovery mnemonic and optional passphrase.
BIP-32 [R15]	Hierarchical derivation reference for recovery authority.
ERC-7739 [R09]	Defensive rehashing of contract signatures; **draft**, review for replay mitigation.
EIP-1193 [R12]	Wallet provider API for websites/extensions.
ERC-20 / ERC-721 / ERC-1155 [R13]	Fungible and NFT balances/permissions.
NIST FIPS 204 [R08]	ML-DSA; research path for post-quantum signatures, not a V1 claim.

Appendix C — Logical Contract Interfaces (Illustrative, Not Deployable)

The following pseudocode is deliberately schematic and is **not Solidity implementation code**. Every signature domain, validation rule, gas condition and error path must be made explicit in audited contracts.

```
KnoadAccount state:
  authorizedHotKey: AddressOrKeyId
  hotGeneration: uint64
  recoveryAuthority: AddressOrKeyId
  recoveryEpoch: uint64
  recoveryNonce: uint64
  orderGrants[digest]: {verifier, expiry, signerRef, orderEpoch, status}

executeAndRotate(calls, expectedGeneration, nextHotKey, signature, nonce):
  require sender is supported EntryPoint
  verify operation signature from current hot key
  require expectedGeneration == hotGeneration
  require full operation is bound to chain/account/EntryPoint/nonce
  require nextHotKey != current hot key and nextHotKey is structurally valid
  execute bounded business calls with defined revert boundary
  if outer execution completes:
      authorizedHotKey = nextHotKey
      hotGeneration += 1
      emit separate action result and rotation result

emergencyReplaceHotKey(nextHotKey, nextRecoveryAuthority,
                       recoveryEpoch, recoveryNonce, signature):
  verify independent recovery signature over exact typed request
  require recovery epoch, nonce, deadline, chain and account match
  atomically set hotGeneration += 1 and authorizedHotKey = nextHotKey
  atomically rotate or explicitly preserve recovery authority by policy
  invalidate stale signature contexts and emit recovery event

isValidSignature(digest, signature) -> bytes4:
  only return ERC-1271 magic value when an exact, unexpired,
  unrevoked, domain-safe grant verifies for authorized protocol context
  otherwise return invalid magic value
```

Explicit prohibited shortcuts: unbounded generic `delegatecall`; unauthenticated `initialize`; active-signer power to alter recovery rules; acceptance of every historic hot signer in ERC-1271; hidden company admin; indexer authorization decisions; blind trust in bundler claims; and public proof signatures from a replacement key before its activation.

Appendix D — Cryptographic Derivation and Restoration Specification

The production artifact must publish exact test vectors for mnemonic encoding and normalization, optional BIP-39 passphrase, recovery root derivation, account index, chain-specific domain separation, initial recovery verifier address, rotated recovery-index derivation, account discovery and signature serialization. This draft commits to following BIP-39 as defined in its standard; it **does not invent arbitrary derivation path numbers** without review. A candidate recovery branch may use hardened BIP-32 derivation, but independent tests must ensure that disclosing public data and non-hardened children cannot expose the recovery root [R06][R15].

Two candidate hot-signing schemes require comparative analysis. **Scheme R (recommended):** fresh random hot keys, encrypted locally, not necessarily deterministically reconstructable; recovery signer independently installs a new hot key on another device. Advantage: the long-lived seed need not be stored on the everyday device. Cost: multi-device signing must be serialized or separately authorized; accidental local loss invokes emergency recovery. **Scheme D (alternative):** deterministic hot keys derived from a hot-specific branch; restores current hot key using onchain generation/index. Advantage: easy local rederivation if correct root remains available. Cost: a stored branch private root can compromise future signers, and a phrase must be available to regenerate branches if not retained. Production implementation must choose one and justify the chosen risk model; mixing them invisibly is prohibited.

Appendix E — Canonical Flow Diagrams

```
NORMAL OPERATION
User action -> policy review -> generate next hot key
 -> sign with current hot key -> EntryPoint submission
 -> account verifies signer + generation + nonce
 -> executes bounded action -> retires current signer
 -> client verifies canonical generation -> activates next key

PENDING / STUCK
Operation pending -> query EntryPoint/account + multiple bundlers
 -> [not included] reprice/replace SAME nonce with consistent next key
 -> [included] check actual result + signer generation
 -> [reorg] rewind provisional state and reconcile

EMERGENCY RECOVERY
Fresh device + recovery phrase [+ optional passphrase]
 -> independently discover smart account -> read recovery epoch
 -> generate new hot key -> sign scoped recovery operation
 -> submit through independent infrastructure
 -> onchain recovery verifier replaces hot signer
 -> recovery authority rotates/updates per finalized protocol

STANDING NFT ORDER
Prepare signed specific marketplace order
 -> authorize exact hash, verifier/scope and expiry
 -> retire everyday signing key on future transaction
 -> later marketplace ERC-1271 check verifies only that order
 -> [expiry/cancel/emergency invalidate] order no longer valid
```

Appendix F — Security Case and Residual-Risk Register

The security case must have three columns in its eventual audit publication: **Claim**, **evidence**, and **remaining limitation**. Examples:

Claim	Required evidence	Non-eliminated limitation
Retired hot key cannot initiate new spends	Property test + audited validation/ execution entry paths	Previously approved spender still may move funds
Owner can recover without KNOAD	Published test vectors and independent recovery demo	Lost phrase or censored chain can prevent recovery
Failed operation cannot strand wallet irrecoverably	All revert/nonce/recovery fuzz branches	Fatal bug in recovery verifier itself
Standing Seaport listing survives normal rotation	Full order-sign/fulfill integration with pinned Seaport	NFT approval, counter or balance can invalidate order
One key cannot bypass another's authority	No hot-key path to recovery controls; reviewed modules	Recovery secret theft supersedes hot policy
No post-transaction public-key persistence risk	Measure authorization window and generation checks	In-flight or recovery ECDSA signature remains quantum-relevant

Risks in Appendix F are **unresolved until evidence is attached**. A source-code review without execution traces does not satisfy those claims.

Appendix G — Verification and Audit Matrix

The release test matrix must map every invariant and threat (Sections 13-14) to exact test names, compiler/tool versions, environment, seed inputs and expected values. Suggested techniques include invariant fuzzing with adversarial calldata, symbolic execution for critical role changes, chain forking against relevant token and marketplace contracts, formal verification of isolated state transitions, gas-liveness testing under maximum bounded calls, and reproducible mobile secret-handling tests. Coverage percentages alone are insufficient: the most valuable evidence is direct proof of authorization and recovery properties under exceptional states.

Critical cases requiring manual adversarial review include ECDSA malleability and signature format, gas refund/reentrancy mechanics of EntryPoint, ERC-1271 replay through cross-chain domains, recovery-nonce treatment during a reverted UserOperation, malformed ABI dynamic data, stale bundle validation, malicious ERC-721 and ERC-1155 callbacks, ERC-20 allowance quirk handling, and factory CREATE2 initialization front-running.

Appendix H — Compatibility and Infrastructure Matrix

For each supported chain, publish: chain ID; EntryPoint deployed version/address; factory and account implementation addresses/code hashes; recovery derivation and deployment indexes; bundler compatibility; paymaster policy; native and token support; EIP-1193 / WalletConnect support; ERC-1271 login compatibility; ENS handling; Seaport version/counter/conduit support; approved order types; Permit2 integration; known token incompatibilities; expected confirmation/finality policy; alternative RPC and independent-recovery steps. Unverified rows must be labeled **not verified**, not silently omitted.

Appendix I — Economics and Performance Measurement Plan

Publish a gas and cost report rather than estimate unmeasured figures. A repeatable benchmark suite must compare basic EOA transfer, ERC-4337 transfer with ordinary validation, ERC-4337 transfer plus hot rotation, standard allowance and scoped allowance plus cleanup, single order registration, cancellation, and emergency replacement. Keep chain base fees, L2 calldata fees, bundler overhead and sponsorship separate. Include at least one adversarial worst-case trace alongside median operations. User-facing fee previews should use current network data, identify KNOAD service fees as zero or explicit, and reject manipulative fee disclosures.

Appendix J — Prototype Disposition and Open Technical Decisions

Prototype examined: `knoad-wallet-security-poc.zip` from the earlier design iteration. Its `RotatingAccount.sol` contains a 130-byte signature path proving both the current and next hot signers, a `generation` check, a bounded `executeAndRotate`, a single-hash ERC-1271 order registry and ERC-721/1155 receiver capabilities. Its factory predicts an address using the initial signer. The repository's own `SECURITY_REVIEW.md` describes it as an uncompiled, unaudited research artifact and points to missing recovery, dApp compatibility and infrastructure testing. This white paper **does not** certify that code or supply deployable replacements.

Decision	Current recommendation	Status
Base smart-account implementation	Reviewed ERC-4337 account core + smallest custom validation policy	Selected approach; implementation outstanding
Hot-key source	Fresh local random keys; do not persist BIP-39 root in daily vault	Preferred candidate; test hardware/storage feasibility
Hot-key proof	No preactivation onchain next-key signature	Security change required
Recovery	Separate user-owned authority; emergency lane; ratcheting after use	Required; contract implementation missing
Recovery material	24 words default; optional strong BIP-39 passphrase	Selected product policy
Account discovery	Independent discovery tied to stable recovery identity	Release blocker; design unresolved
Factory initialization	Authenticated, front-run-resistant	Release blocker; design unresolved
Rotation on reverted business calls	Preserve rotation when defined outer operation succeeds; report action result separately	Candidate; EntryPoint execution testing missing
ERC-1271 order durability	Exact digest, limited scope, expiry and revocation	Candidate; marketplace compatibility unverified
Owner-controlled upgrades	Prefer minimal core, rigorously assessed upgrade/migration	Open decision
Post-quantum signatures	Separate future cryptographic-agility migration	Research roadmap, not V1 security claim

P0 decisions to resolve before implementation freeze: precise signer-key storage scheme; front-run-safe factory/address restoration; independent recovery epoch and next-recovery-key proofs; exact transaction-revert/rotation semantics; Seaport digest and ERC-1271 context binding; upgrade/migration trust model; and recovery client independence. The white paper is complete as a proposed technical blueprint, not a complete audited specification until these implementation-specific choices and their proofs are appended.

Sources and Standards

- **[R01]** ERC-4337, *Account Abstraction Using Alt Mempool*. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-4337
- **[R02]** ERC-1271, *Standard Signature Validation Method for Contracts*. https://eips.ethereum.org/EIPS/eip-1271
- **[R03]** ERC-7579, *Minimal Modular Smart Accounts* (**draft**). https://eips.ethereum.org/EIPS/eip-7579
- **[R04]** Ethereum.org, *EIP-7702 delegation and security guidelines*. https://ethereum.org/roadmap/pectra/7702/
- **[R05]** OpenZeppelin, *Contracts 5.x: Account and Account Abstraction*. https://docs.openzeppelin.com/contracts/5.x/accounts
- **[R06]** BIP-39, *Mnemonic Code for Generating Deterministic Keys*. https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki
- **[R07]** Ethereum.org, *Post-Quantum Cryptography on Ethereum*. https://ethereum.org/roadmap/security/quantum-resistance/
- **[R08]** NIST, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, 2024. https://csrc.nist.gov/pubs/fips/204/final
- **[R09]** ERC-7739, *Readable Typed Signatures for Smart Accounts* (**draft**). https://eips.ethereum.org/EIPS/eip-7739
- **[R10]** Project OpenSea, *Seaport Documentation*. https://github.com/ProjectOpenSea/seaport/blob/main/docs/SeaportDocumentation.md
- **[R11]** OWASP, *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- **[R12]** EIP-1193, *Ethereum Provider JavaScript API*. https://eips.ethereum.org/EIPS/eip-1193
- **[R13]** ERC-20 / ERC-721 / ERC-1155 token standards. https://eips.ethereum.org/EIPS/eip-20 ; https://eips.ethereum.org/EIPS/eip-721 ; https://eips.ethereum.org/EIPS/eip-1155
- **[R14]** NIST SP 800-63B, *Authentication and Authenticator Management*. https://pages.nist.gov/800-63-4/sp800-63b.html
- **[R15]** BIP-32, *Hierarchical Deterministic Wallets*. https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki

Source cutoff: Retrieved and reviewed 13 October 2026. Standards labeled draft may change; code must pin exact revisions and implementations before audit. All proprietary roadmap, performance, security and release statements are forward-looking design commitments unless accompanied by independent validation.